

Sesión 2 · Práctica 2.2 · Lógica de negocio con Power Fx

Fórmulas, condicionales, variables y colecciones

Dato	Valor
Duración	50 minutos
Nivel	Inicial-intermedio
Archivos	Ninguno
Resultado	App de dos pantallas: calculadora de presupuesto con formato condicional y gestor de ciudades con colecciones, filtros y Patch

Objetivo

Power Fx es el lenguaje de fórmulas de Power Apps, y se parece mucho a las fórmulas de Excel: cada propiedad de cada control puede tener una fórmula que se recalcula sola cuando cambian los datos de los que depende. En esta práctica vas a trabajar las piezas de lógica que usarás en cualquier aplicación: operaciones y formato de números, condicionales con `If` y `Switch`, variables globales y de contexto, y colecciones para guardar tablas en memoria.

Construirás una app en blanco con dos pantallas independientes. En la primera, una calculadora de presupuesto. En la segunda, un pequeño gestor de ciudades que se guarda en una colección.

Notación. Todas las fórmulas usan `;` como separador de argumentos y `;;` para encadenar instrucciones (navegador en español).

Parte 1 · Calculadora de presupuesto

1. Crea una **aplicación de lienzo en blanco** en formato tableta llamada `Lógica Power Fx - <iniciales>`.
2. Renombra `Screen1` como `scrCalc` y añade una cabecera con el texto `"Presupuesto"`.
3. Inserta cuatro etiquetas con los textos `Producto`, `Cantidad`, `Precio unitario` y `Total`, una debajo de otra.
4. A la derecha de las tres primeras, inserta tres **cuadros de texto** y llámalos `txtProduct`, `txtQty` y `txtPrice`. Borra su texto predeterminado para que empiecen vacíos.
5. Pon `txtQty` y `txtPrice` en formato **Número** desde el panel de propiedades.
6. A la derecha de `Total`, inserta una etiqueta `lblTotal` con esta fórmula en **Text**:

```
Text(Value(txtQty.Text) * Value(txtPrice.Text); "#.##0,00 €")
```

1. Prueba en vista previa: cantidad 4 , precio 2,30 . Debe mostrar 9,20 € .

Por qué Value() . Un cuadro de texto siempre devuelve texto, aunque lo hayas puesto en modo número. Value() lo convierte en número para poder multiplicar. Escribir Sum(TextInput2 * TextInput3) multiplicaría controles en lugar de sus valores y daría error.

1. Añade una etiqueta ancha lblSummary con un resumen que se construye concatenando texto con & :

```
"El presupuesto de " & txtProduct.Text & " asciende a " & lblTotal.Text
```

Parte 2 · Formato condicional

1. Selecciona lblTotal y cambia su propiedad **Color** para que el importe se vea en verde si supera 50 € y en azul si no:

```
If(Value(txtQty.Text) * Value(txtPrice.Text) > 50; Green; Blue)
```

1. Añade una etiqueta lblLevel que clasifique el presupuesto en tres tramos con Switch sobre un valor verdadero:

```
Switch(  
    true;  
    Value(txtQty.Text) * Value(txtPrice.Text) >= 1000; "Requiere aprobación de dirección";  
    Value(txtQty.Text) * Value(txtPrice.Text) >= 100; "Requiere aprobación del  
responsable";  
    "Aprobación automática"  
)
```

1. Observa que estamos repitiendo el mismo cálculo varias veces. Sustitúyelo usando With , que da un nombre local a un valor dentro de una fórmula:

```
With(
    { total: Value(txtQty.Text) * Value(txtPrice.Text) };
    Switch(true; total >= 1000; "Requiere aprobación de dirección"; total >= 100;
    "Requiere aprobación del responsable"; "Aprobación automática")
)
```

1. Añade un icono de **avión** y haz que su posición vertical dependa de un desplegable. Inserta un desplegable `drpPosition` con **Items** ["Arriba"; "Centro"; "Abajo"] y pon en la propiedad **Y** del icono:

```
Switch(drpPosition.Selected.Value; "Arriba"; 150; "Abajo"; 600; 380)
```

Parte 3 · Variables de contexto y globales

1. Inserta un cuadro de texto `txtAmount` en modo número y dos botones: `btnAdd` con texto `Sumar` y `btnReset` con texto `Poner a cero`.
2. En **OnSelect** de `btnAdd`, acumula el valor en una **variable de contexto**, que solo existe en esta pantalla:

```
UpdateContext({ locRunningTotal: locRunningTotal + Value(txtAmount.Text) });;
Reset(txtAmount)
```

1. En **OnSelect** de `btnReset`: `UpdateContext({ locRunningTotal: 0 })`.
2. En **DisplayMode** de `btnReset`: `If(locRunningTotal > 0; DisplayMode.Edit; DisplayMode.Disabled)`.
3. Añade una etiqueta con **Text** `"Acumulado: " & Text(locRunningTotal; "#.##0,00 €")` y pruébalo.
4. Ahora una **variable global**, visible desde cualquier pantalla. En **OnSelect** de un botón `btnSaveBudget` con texto `Guardar presupuesto`:

```
Set(gblLastBudget; { Producto: txtProduct.Text; Total: Value(txtQty.Text) *
Value(txtPrice.Text); Fecha: Now() })
```

1. Abre el panel **Variables** del menú de creación y comprueba el contenido de `locRunningTotal` y `gblLastBudget`.

Cuándo usar cada una. `UpdateContext` para estados de una sola pantalla (un contador, si un panel está abierto). `Set` para datos que necesitas en varias pantallas (el usuario, el último registro elegido). Y si el valor se puede calcular siempre a partir de otros, no uses variable: usa una fórmula con nombre en `App.Fórmulas`, que veremos en la Sesión 3.

Parte 4 · Colecciones

1. Crea una pantalla nueva `scrCities` y un botón de navegación hacia ella desde `scrCalc`.
2. En la propiedad **OnVisible** de `scrCities`, crea una colección con datos de ejemplo:

```
ClearCollect(
    colCities;
    { City: "Madrid"; Country: "España"; Population: 3332000 };
    { City: "Barcelona"; Country: "España"; Population: 1660000 };
    { City: "Valencia"; Country: "España"; Population: 807000 };
    { City: "Lisboa"; Country: "Portugal"; Population: 545000 };
    { City: "Oporto"; Country: "Portugal"; Population: 232000 };
    { City: "Berlín"; Country: "Alemania"; Population: 3878000 };
    { City: "Múnich"; Country: "Alemania"; Population: 1510000 };
    { City: "Roma"; Country: "Italia"; Population: 2750000 };
    { City: "Milán"; Country: "Italia"; Population: 1370000 }
)
```

1. Para ejecutarla mientras editas, sal de la pantalla y vuelve a entrar en vista previa, o pulsa los puntos suspensivos de la pantalla y elige **Ejecutar OnVisible**.
2. Inserta una **galería vertical** `galCities` con **Items** `colCities`, diseño *Título, subtítulo y cuerpo*, y en el cuerpo `"Población: " & Text(ThisItem.Population; "#.##0")`.

Parte 5 · Filtrar, buscar y ordenar

1. Inserta un **control deslizante** `sldPopulation` con **Min** `Min(colCities; Population)` y **Max** `Max(colCities; Population)`.
2. Inserta un desplegable `drpCountry` con **Items** `Distinct(colCities; Country)`.
3. Cambia **Items** de la galería para combinar ambos filtros y ordenar por población:

```
SortByColumns(
    Filter(colCities; Population >= sldPopulation.Value && Country =
drpCountry.Selected.Value);
    "Population"; SortOrder.Descending
)
```

1. Añade una etiqueta con el número de ciudades que se están mostrando:
`CountRows(galCities.AllItems) & " ciudades"`.

Parte 6 · Leer y modificar un registro con LookUp y Patch

1. Inserta tres cuadros de texto `txtCity`, `txtCountry` y `txtPop`, con sus etiquetas.

2. En **OnSelect** de la galería, guarda el registro elegido en una variable:

```
Set(varCity; ThisItem)
```

1. Pon en la propiedad **Default** de cada cuadro: `varCity.City`, `varCity.Country` y `varCity.Population`.
2. Inserta cuatro botones: `Guardar`, `Nuevo`, `Eliminar` y `Limpiar`, con estas fórmulas en **OnSelect**:

Botón	OnSelect
Guardar	<code>Patch(colCities; varCity; { City: txtCity.Text; Country: txtCountry.Text; Population: Value(txtPop.Text) });; Set(varCity; Blank())</code>
Nuevo	<code>Collect(colCities; { City: txtCity.Text; Country: txtCountry.Text; Population: Value(txtPop.Text) });; Set(varCity; Blank())</code>
Eliminar	<code>Remove(colCities; varCity);; Set(varCity; Blank())</code>
Limpiar	<code>Set(varCity; Blank())</code>

1. En **DisplayMode** de `Guardar` y `Eliminar`: `If(IsBlank(varCity); DisplayMode.Disabled; DisplayMode.Edit)`.
2. Busca una ciudad concreta con `Lookup`: añade una etiqueta con `"La ciudad más poblada es " & First(SortByColumns(colCities; "Population"; SortOrder.Descending)).City`.
3. Prueba a modificar la población de Valencia, crear Sevilla y eliminar Oporto.

Colecciones y datos reales. Una colección vive en la memoria del dispositivo y desaparece al cerrar la app. Es perfecta para datos temporales, para un carrito o para trabajar sin conexión, pero no sustituye a un origen persistente como la tabla de Excel en OneDrive o una tabla de Dataverse. La misma función `Patch` que acabas de usar funciona igual contra la tabla de Excel, como viste en la Práctica 2.1.

Comprobación

- La calculadora muestra el total con formato de euros, cambia de color y clasifica el presupuesto en tres tramos.
- El acumulado funciona con una variable de contexto y el botón de poner a cero se deshabilita cuando está en cero.
- La variable global `gblLastBudget` aparece en el panel de variables.
- La galería de ciudades filtra por población y país, y ordena por población.
- Se pueden añadir, modificar y eliminar ciudades en la colección.

Retos opcionales

1. Valida el correo electrónico: añade un cuadro `txtEmail` y un botón que solo se habilite si el correo tiene formato válido con `IsMatch(txtEmail.Text; Match.Email)`.
2. Añade una columna calculada a la galería con `AddColumns(colCities; Densidad; ...)` o muestra la población en millones con `Round(Population / 1000000; 2)`.
3. Guarda la colección en el dispositivo con `SaveData(colCities; "ciudades")` y recupérala con `LoadData(colCities; "ciudades"; true)` en la aplicación de Power Apps Mobile.